
Learning to Play Mini-Quoridor with AlphaZero-Style Self-Play

June 11, 2026

Riccardo Calderoni

Abstract

This experiment investigates whether an AlphaZero-style reinforcement learning pipeline can learn to play a smaller version of the board game Quoridor. In Quoridor, two players race to reach the opposite side of the board while placing walls to slow down the opponent. The agent is trained from self-play using Monte Carlo Tree Search guided by a convolutional policy-value network. Starting from an untrained policy, the agent learns movement and wall-placement strategies through repeated self-play games. The experiments focus on how search and exploration choices influence learning stability, game length, and stalled positions. The final agents exhibit meaningful strategic behavior. Code: <https://github.com/rikcald/Quoridor-AI>

1. Introduction

Quoridor is a two-player deterministic board game in which each player must reach the opposite side of the board while using a limited number of walls to delay the opponent. This combination of path planning and blocking makes the game interesting for reinforcement learning: good play requires both short-term tactical movement and longer-term spatial reasoning about possible routes. In this project, I study a reduced version of the game, played on a 5×5 board with four walls per player, instead of the classical 9×9 board with ten walls per player. Despite the reduced board size, "Mini-Quoridor" remains combinatorially non-trivial. A rough upper bound gives approximately 2×10^{12} raw board configurations for the 5×5 version with four walls per player. By comparison, the full 9×9 game has been estimated at around 4×10^{42} configurations (Mertens, 2006), close to the scale commonly reported for chess. The reduced setting therefore keeps the main strategic structure of the original game while making self-play training feasible on limited hardware. The goal of the project is not to reproduce the full-scale results of AlphaZero, but to implement the main ideas of the algorithm from scratch and test whether they can produce meaningful results in a smaller domain. The system includes a

complete game environment, legal move generation, neural network inference, Monte Carlo Tree Search, self-play data generation, checkpointing, training metrics, and arena evaluation between trained agents. The experiments focus especially on the interaction between search depth, exploration, temperature scheduling, and stalled games.

2. Related work

This project is inspired by AlphaZero, introduced by Silver et al. as a general reinforcement learning algorithm able to learn chess, shogi, and Go from self-play, without human expert games or handcrafted evaluation functions (et al., 2017). The implementation also follows Surag Nair’s AlphaZero General framework (Nair, 2017), which presents the algorithm as a policy iteration loop. In each self-play position, MCTS produces an improved move distribution from visit counts; after the game ends, the final outcome is used as the value target. The network is then trained to imitate the MCTS policy and predict the game result.

3. Method

The environment implements Mini-Quoridor on a 5×5 board. Each player starts on the opposite side and wins by reaching the other side. The action space contains 48 possible action slots: 16 movement actions, including normal moves, jumps, and diagonal jumps, plus 32 wall-placement actions corresponding to horizontal and vertical walls. Illegal actions are masked before sampling or training. Wall placement is only allowed if both players still have at least one valid path to their goal. The neural network is a convolutional policy-value model implemented in PyTorch. The input is a canonical player-centric board representation with six planes: current player position, opponent position, horizontal walls, vertical walls, current player remaining walls, and opponent remaining walls. The shared trunk consists of three convolutional blocks with batch normalization and ReLU activations. Two branches are then used: a policy head producing logits over the 48 actions, and a value head producing a scalar in $[-1, 1]$, representing the expected outcome from the current player’s perspective. At every move during self-play, MCTS is run from the current state. The tree uses the PUCT rule (et al., 2017) to select

which branches to expand:

$$Q(s, a) + c_{puct} P(s, a) \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}$$

$Q(s, a)$ is the estimated value of an action, $P(s, a)$ is the neural network prior, $N(s, a)$ is the visit count, and the constant c_{puct} controls the exploration–exploitation trade-off in MCTS. There also is a Dirichlet noise that is added at the root during self-play to encourage exploration. After search, the root visit counts are converted into a target policy. The action temperature controls how the move is sampled, while a separate target-policy temperature controls how sharp the policy target is during training. This separation avoids training the network only on one-hot targets when self-play becomes greedy after the opening moves. Training examples have the form (s_t, π_t, z_t) , where s_t is the canonical state, π_t is the MCTS visit-count distribution, and z_t is the final outcome from the perspective of the player who acted in that state. The network is optimized with the sum of policy cross-entropy loss and value mean squared error. Because some self-play games can reach the maximum step limit without a terminal winner, timeout adjudication is used: Unfinished games are assigned a weak value target based on which player has the shorter legal path to the goal.

4. Experimental Results

The experiments were organized in two stages: an initial training run from an untrained network, followed by several fine-tuning runs from selected checkpoints. The first long run trained for 3500 self-play games with 200 MCTS simulations per move, producing the first functional baseline agent. Figure 1 shows rolling averages for game length, loss, and wall usage during this run. The average number of steps and wall usage both decrease over training, suggesting that the agent gradually learns more direct strategies and becomes more selective with wall placement. The policy loss drops quickly and then stabilizes, while the value loss slightly increases, likely because stronger self-play generates more balanced and harder-to-evaluate positions. Starting from this baseline, I tested several fine-tuning configurations to reduce stalled games and improve playing stability, adjusting hyperparameters such as the temperature schedule, exploration noise, c_{puct} , and number of MCTS simulations. Table 1 summarizes the most informative self-play runs. The $c_{puct} = 3.5$ run achieved the lowest timeout rate and shortest average games during training, while the high-simulation run obtained the lowest loss but worse gameplay statistics, so it was stopped early due to its much higher computational cost. This shows that a lower training loss does not necessarily correspond to a stronger agent. To compare the agents more directly, I also

Table 1. Comparison of selected self-play runs.

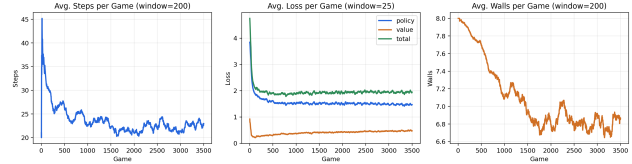
Run	Simulations	games	c_{puct}	Timeout	Avg. steps	Loss
Baseline	200	3500	1.5	5.5%	23.3	1.95
Balanced fine-tune	600	500	1.5	3.4%	21.9	1.64
Low expl.	600	500	0.7	10.2%	27.6	1.57
Medium expl.	600	500	2.5	3.0%	21.1	1.77
High expl.	600	500	3.5	2.6%	20.6	1.75
High sim.	10000	144	1.5	15.3%	30.7	1.45

Table 2. Tournament arena results between the main checkpoints

Model	Wins	Losses	Draw/Timeout	Win rate
Balanced	582	220	98	72.6%
Medium expl.	421	371	108	53.2%
Baseline	334	508	58	39.7%
High expl.	259	497	144	34.3%

evaluated the main checkpoints in a head-to-head tournament. Table 2 shows that the self-play statistics were partly misleading: the $c_{puct} = 3.5$ model was not the strongest agent in head-to-head play, and often lost even against the baseline. The best overall arena result came from the intermediate fine-tuned model. This indicates that self-play stability is useful for diagnosing training, but final model quality should be judged through direct evaluation against other agents.

Figure 1. Baseline plot



5. Discussion and conclusion

The results show that an AlphaZero-style pipeline can learn meaningful Mini-Quoridor strategies from self-play. Even with a small convolutional network and a limited number of games, the agent learned both movement and wall-placement behavior without human demonstrations. At the same time, the head-to-head tournament highlights a practical limitation: self-play timeout rate and training loss are useful diagnostics, but they are not reliable proxies for playing strength. The remaining failures mostly occur when agents are heavily blocked and must find long detours, suggesting that deeper convolutional blocks or additional path-related feature planes could improve spatial reasoning, at the cost of higher computation, also continuing training from the best fine-tuned checkpoint with its hyperparameters would likely improve the agent further. Overall, the project provides a working proof of concept: AlphaZero’s core ideas can be implemented from scratch and adapted to Mini-Quoridor, producing agents with recognizable strategic behavior through self-play.

6. Statement on the use of AI

AI-based tools were used as support during the development of this project. In particular, they assisted with code debugging, report editing, the generation of auxiliary plotting scripts, code commenting and organization, and the refactoring of some Python script functions. They were also used for brainstorming possible hyperparameters to test during fine-tuning experiments.

All code, experiments, results, and final writing were reviewed and verified by the author. No code or technical content was included in the final submission without being understood, checked, and accepted by the author.

References

- et al., D. S. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. URL <https://arxiv.org/abs/1712.01815>.
- Mertens, P. J. C. A quoridor-playing agent, 2006. URL https://project.dke.maastrichtuniversity.nl/games/files/bsc/Mertens_BSc-paper.pdf.
- Nair, S. A simple alpha(go) zero tutorial, 2017. URL <https://suragnair.github.io/posts/>.